DATASCI 3ML3 Final Project

Wikipedia Article Classification

Liam Strilchuk*

April 20, 2026

Contents

1 Overview and Motivation **2**

2 Data Acquisition and Preprocessing **2**

3 Feature Engineering **3**

4 Model Implementation **4**

5 Evaluation and Insights **5**
5.1 Experiment: Data Volume 5
5.2 Experiment: Dropout 6
5.3 Experiment: Max Features 7
5.4 Experiment: Model Size 8
5.5 Final Model 8

6 Conclusion **9**
6.1 Next Steps 9

7 Code Organization **10**

8 References **10**

*strilchl@mcmaster.ca

1 Overview and Motivation

On Wikipedia, before articles are made visible to the public, they must go through a review process. Reviewers, who are volunteers, must follow a 30+ step flowchart for each published article;[1] this time-consuming process, along with a shortage of reviewers, means there is often a months-long waiting time for articles to be reviewed (at the time of writing, the number of articles waiting to be reviewed is over 17,000). Therefore, any additional tools that could shorten the time taken to review an article would be incredibly beneficial to the Wikipedia community.

For this project, we will create a model which can handle a step of this process. Articles can be classified into one or more *WikiProjects*; a WikiProject is a group of editors focusing on a specific topic. For example, WikiProject Canada is a group of (mainly Canadian) editors improving Canada-related articles. Classifying articles into projects is extremely important, since it allows them to be targeted for improvement by editors knowledgeable about the subject area; unclassified articles are often neglected. However, since classifying is considered an "optional" step in the reviewing process, many articles end up unclassified.

To this end, we will create a model to solve this multi-label classification problem. This is different from multi-class classification, since an article can be part of zero, one, or more WikiProjects. Since there are thousands of projects of varying size (for example, *Computing* has 36,000 articles; *Artificial intelligence* has 1,600; and *Human-computer interaction* has only 184), we will focus on classifying only the approximately 25 largest projects as a proof-of-concept. However, we will explore next steps for a full model in the conclusion.

We will use Tensorflow/Keras to build and train the model, along with `scikit-learn` for its natural language processing features (specifically, TF-IDF). In the remainder of this report, we will discuss data acquisition and processing, creating a model, evaluation and experimentation; finally, we will create an improved model based on these insights.

2 Data Acquisition and Preprocessing

To train our model, we need data in the format $(lead, projects)$, where the *lead* is the beginning article text, and *projects* is a list containing the WikiProjects that article has been assigned. While there is no existing dataset containing the exact data we need for this model, Wikipedia is very open and provides a number of APIs which we can use to collect the data ourselves. We will collect only the lead paragraphs of an article, instead of its full content, as its projects can be determined from the first paragraph alone in the vast majority of cases.

Wikipedia provides a directory of all WikiProjects, along with details about their size.[2] To simplify the model, we will classify only the 25 WikiProjects with the largest number of articles. Our first goal is to find a list of articles in each project; we can use the WP 1.0 API for this.

A major problem that we have to deal with here is the relative sizes of each category. WikiProject Biography, for example, has nearly 2.2 million articles; while a smaller WikiProject Lepidoptera has only around 100,000. This difference would be even more pronounced if we were to classify more than 25 projects: the vast majority of projects have fewer than 10,000 articles. If we were to use all possible data to train our model, it would likely have a bias towards larger projects while potentially ignoring smaller ones.[3]

To combat this, we use a technique called undersampling. While there are fairly complex algorithms that can be used for this purpose, we will use the simpler *random undersampling*, which will result in a more balanced dataset. The way this works in practice is by training on only a portion of the available dataset for majority classes. So, we will obtain only a random subset of articles in each project: approximately 2,000 from each. This is likely far more than needed to train an effective model; in the Evaluation section below, we will explore training models with varying amounts of available data.

After finding a list of the articles from each project, we will use Wikipedia's `extracts` API to fetch the lead section for each. Finally, since an article can be part of more than one project, we go through each article and determine its other projects. Unfortunately, Wikipedia does not provide a convenient API for this purpose; so instead, we use regular expressions to parse the markup code for each article. We then save all of our collected data to `all_data.csv`. An example row in this dataset is the following:

[1] See `https://upload.wikimedia.org/wikipedia/commons/f/f4/NPP_flowchart.svg`.

[2] See `https://en.wikipedia.org/wiki/Wikipedia:WikiProject_Directory/All`.

[3] O. Olawale Awe, "Computational Strategies for Handling Imbalanced Data in Machine Learning"

```
Treaty of London (1867),"The Treaty of London (French: Traite de Londres), often
called the Second Treaty of London after the 1839 Treaty, granted Luxembourg full
independence...",WikiProject London|WikiProject Luxembourg|...
```

3 Feature Engineering

At the core of our model is the TF-IDF (term frequency-inverse document frequency) algorithm. Unlike the simpler bag-of-words model, which only captures the frequency of a term over a dataset, TF-IDF measures the relative importance of each term. We define $f_{t,d}$ as the number of times term t occurs in document d; we then define the *term frequency* as

$$\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Or, in simpler terms, TF represents the fraction of words that are t in d. The other metric we need to consider is *inverse document frequency*. Let D be the set of all documents in our dataset:

$$\mathrm{IDF}(t,D) = \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

This measures the relative rarity of words across the dataset: higher values means the term occurs in fewer documents. A logarithm is used because without it, a word that appears once in a million documents would have an IDF score of 1,000,000, resulting in the algorithm weighing it much higher than it should. We then multiply the TF and IDF values together to get the final result:

$$\text{TF-IDF}(t,d,D) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t,D)$$

For example, consider the word "the". As one of the most commonly used words in English, it appears in approximately 96% of the leads in our dataset; this would result in its IDF score being $\log \frac{100}{96} \approx 0.02$, or an extremely low "importance". However, if we consider the word "Canadian", which only appears in 4% of leads, its IDF would be $\log \frac{100}{4} \approx 1.4$.

For our model, we will use the `scikit-learn` library's `TfidfVectorizer` class. This has a number of parameters to improve the algorithm's accuracy and performance:

- `max_features` limits the number of terms, selecting only the top N features by their term frequency. Setting this limit reduces the dimensionality of the data, which can reduce memory usage and prevent overfitting by excluding rare words.
- `stop_words` excludes common English words which do not provide a significant amount of information, helping the model focus on more important terms. This can also have the benefit of reducing the number of features and thus improving performance.
- `ngram_range` is a tuple (min, max) that tells the vectorizer how many tokens in a row to consider as a single term. For example, with the input "New York City", the range $(1,1)$ would produce `["New", "York", "City"]`, whereas $(1,3)$ would also consider `"New York"` and `"New York City"`, allowing for more semantic context. However, this comes at the cost of a significantly higher number of features.

Importantly, when fitting the vectorizer to the data, we use only the training data, and not any test data, to ensure there is no data leakage. To begin with, we will set the vectorizer to have a maximum of 10,000 parameters and an ngram range of $(1,2)$ to capture some semantic context.

We also have to convert our list of WikiProjects for each article into something the model can understand: for this, we use `scikit-learn`'s `MultiLabelBinarizer`. This takes a list of strings (eg. `["WikiProject ABC", "WikiProject DEF"]`) and converts it into a multi-hot encoded vector (`[1, 0, ..., 1]`).

4 Model Implementation

We implement our model as a deep feed-forward neural network using Keras's `Sequential` model. Our goal is to create a "funnel" where thousands of input features are mapped using hidden layers into much denser semantic predictions. Our input layer varies in size depending on the number of features given by the vectorizer. We then choose an arbitrary but reasonable value for the size of the first hidden layer: 512. The second hidden layer then continues the "funnel" with half the number of neurons compared to the first. Finally, the output layer matches the number of WikiProjects we are categorizing into.

For both of the dense layers, we use the ReLU activation function as it prevents vanishing gradients and allows the network to learn non-linear relationships. The output layer uses the Sigmoid activation function, which calculates a probability between 0 and 1 for each node (instead of Softmax, which sums all outputs to 1 and is used for multiclass classification). We use binary cross-entropy, which uses the following formula, as our cost function:

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}(y_1 \log(p_i) + (1 - y_i)\log(1 - p_i))$$

This function independently calculates the loss for each label, which allows the model to predict multiple labels for a single input. It has the effect of heavily punishing confidently incorrect predictions. Finally, we use the Adam optimizer, which adjusts the learning rate for each parameter and can lead to faster convergence than normal gradient descent.

To prevent overfitting, we use Dropout regularization layers following each of the Dense layers. This randomly "turns off" neurons with a specified frequency, forcing the model to learn patterns instead of memorizing training data. We initally set the dropout probability to 0.3, though we will experiment with different dropout rates in Section 5.2 below.

For training the model, we split the dataset into a training set (with 80% of the data) and a testing set (20%). When training using `model.fit`, 10% of the training data is used for validation, alongside Keras's `EarlyStopping` callback. This callback monitors the validation loss; once this increases and begins to diverge from the training loss, we know that the model has begun to overfit and we stop training. The parameters we use for this are `patience=2` (so `val_loss` needs to increase over two consecutive epochs before stopping) and `restore_best_weights=True`, which ensures we use the weights from the epoch with the lowest loss, instead of the final, slightly overfitted weights.

5 Evaluation and Insights

In this section, we run a number of experiments with the goal of determining the optimal hyperparameters for our model. Creating a larger or more "powerful" model comes at the cost of additional resource consumption (eg. more memory or computing time) while in many cases not improving performance.

5.1 Experiment: Data Volume

Since our model is classifying no more than the 100 largest WikiProjects, all of which have at least 20,000 articles, our model is not suffering from a lack of training data. However, should the model be expanded in the future to much smaller projects, some of which have only hundreds of examples, this could become a problem. Therefore it would be beneficial to determine how much data is needed to make accurate predictions.

Figure 1 shows the results of the experiment. The chart shows the classification metrics (Precision, Recall, and F1-Score) graphed against the portion of the dataset used for training: between 1% and 90%. To ensure a fair comparison between these radically different dataset sizes, instead of artificially constraining the number of epochs, the Keras callback `EarlyStopping` was used to stop training when the validation accuracy decreased. This ensures that models with less data do not overfit due to their limited sample size.

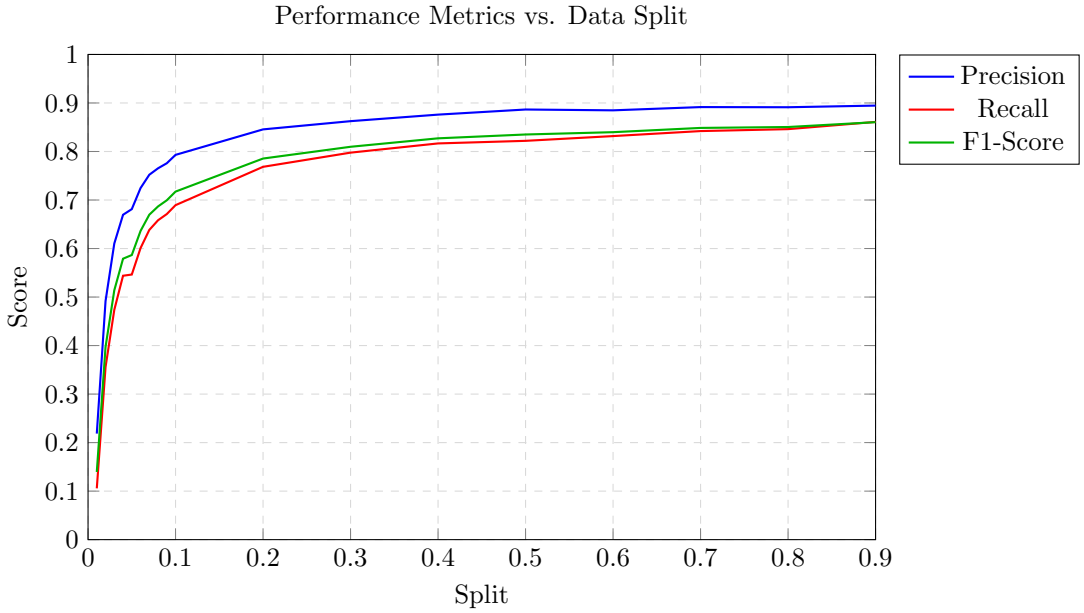

Figure 1: Precision, Recall, and F1-score evaluated across different data splits.

While the model's performance when using less than 10% of the dataset is extremely poor, the accuracy begins to level out around 20%, and by 50% we experience diminishing returns and any improvement becomes very marginal. While increasing the amount of data used does result in an improvement, even if small, it comes at a cost: the training time increases significantly when more data is used. For example, when all available data was used to train a model, the training time was roughly 3x that of a model using only 30%; however, the improvement in accuracy was only 2%. Thus it may not be worth it to always use the largest possible dataset.

5.2 Experiment: Dropout

Dropout is a regularization technique used in neural networks to prevent overfitting. It entails deactivating a random portion of neurons when training, forcing the network to learn robust patterns instead of memorizing the training data. In this experiment, we train models with different levels of dropout and examine their training and validation loss throughout the process. This experiment uses models that have low, medium, and high levels of dropout (0.0, 0.3, and 0.6 respectively).

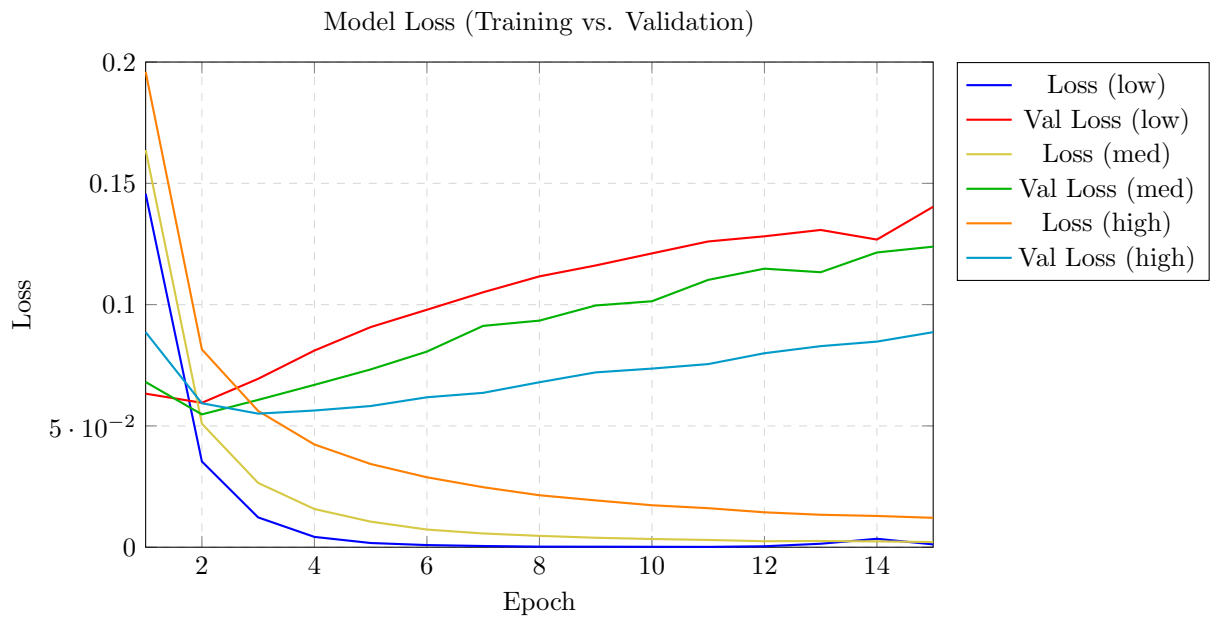

Figure 2: Training and validation loss across epochs for different dropout rates.

Figure 2 shows the results of the experiment. As expected, the model with no dropout has its training loss quickly drop to near zero while its validation loss steadily rises, indicating overfitting. While the medium and high dropout rates smooth this curve and delay the divergence, they fail to prevent the validation loss from increasing. All models achieve their absolute lowest validation loss very early, around epoch 2-3, showing the importance of the early stopping technique. Since even a high level of dropout fails to prevent overfitting, this suggests that the model possesses excess capacity, and reducing the size of the network's layers may be more effective at preventing overfitting.

5.3 Experiment: Max Features

The `max_features` parameter of the `TfidfVectorizer` class was discussed in Section 3. Optimizing its value can significantly impact both the training time and performance of a model. Too low, and the model does not have enough data to accurately classify text; too high, and the memory usage and training time increase, and we risk providing the model with thousands of "filler" words which do not matter for classification.

This experiment tests the performance metrics of models based on the maximum number of features. Figure 3 below shows the results.

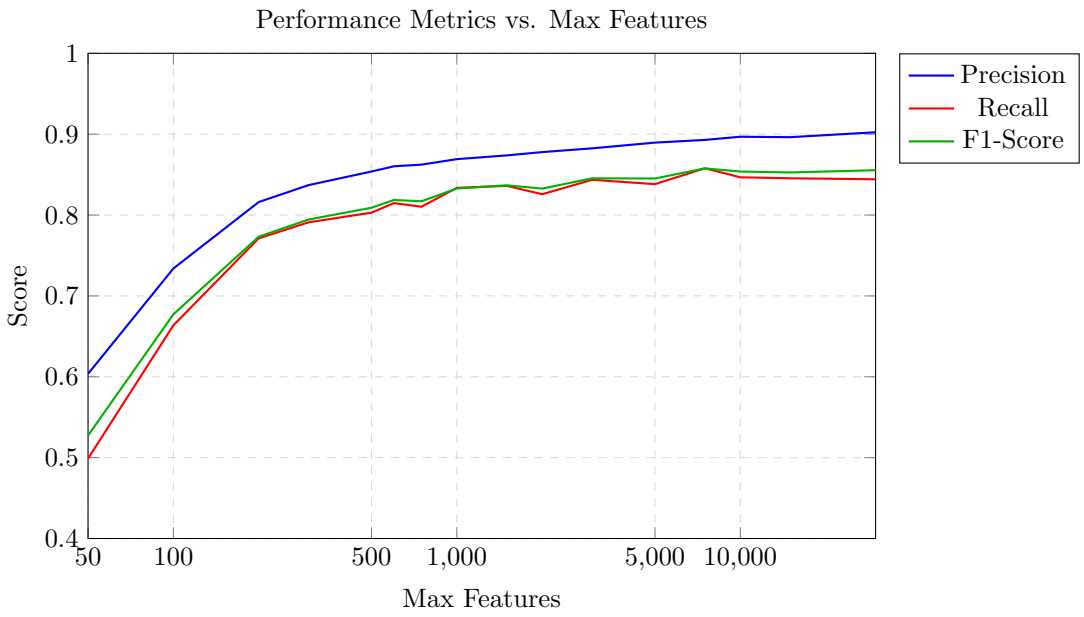

Figure 3: Performance metrics evaluated across a count of max features (logarithmic scale).

Even 100 features is enough to achieve a reasonable degree of accuracy. The increase in model performance becomes marginal once the number of features reaches 1,000; after 10,000, there is no improvement whatsoever. This is likely because Wikipedia articles, despite covering a vast number of topics, can be categorized using a relatively small vocabulary: the words "album" or "Canadian" can tell us, for example, that an article should be put into the "Albums" or "Canada" WikiProjects respectively, without a large amount of other context needed.

Modifying the maximum number of features comes with a significant increase in the number of trainable parameters in the model, and therefore can increase its training time. For example, increasing the feature space from 100 features to 10,000 results in a 700% jump in training time.

5.4 Experiment: Model Size

Finally, we will run an experiment comparing models with different sizes of hidden layers. We measure the layer size as a factor of our arbitrarily picked initial values of 512 and 256 for the first and second layers respectively. The more neurons a network has, the more capacity it has for learning complex patterns; however, this comes at the cost of increased memory usage, training time, and potential for overfitting. For example, a network with dense layers of sizes 1,024 and 512 has 524,288 trainable parameters between them; if the sizes are reduced by a factor of 8 to 128 and 64, this lowers to only 8,192.

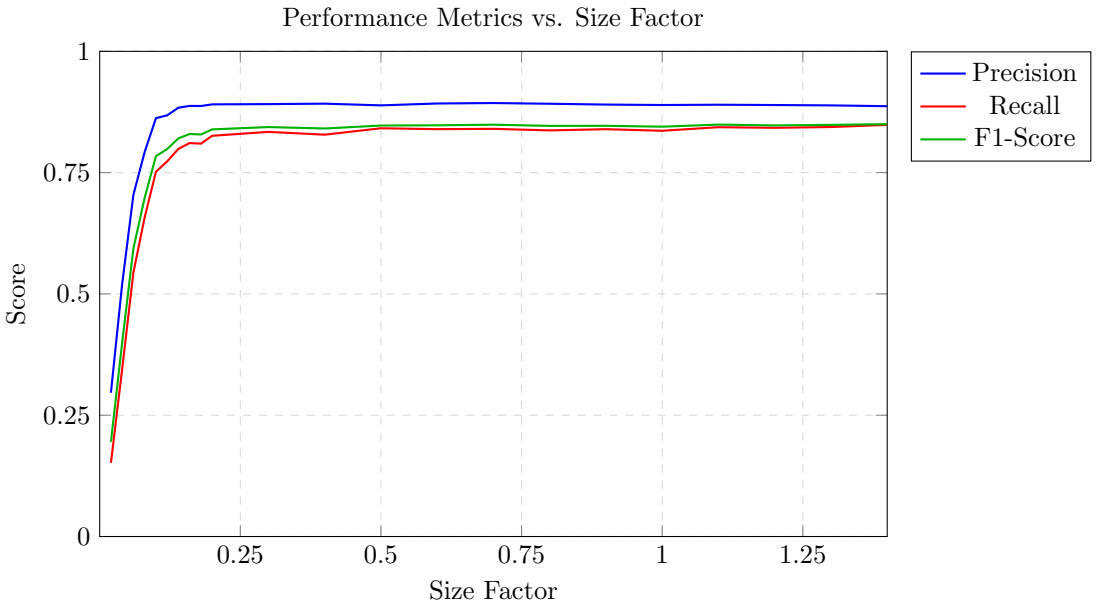

Figure 4: Performance metrics evaluated across different model size factors.

Figure 4 shows the results of this experiment. We can see that our initial estimation of 512/256 was significantly more than needed. The performance metrics increase sharply until a factor of 0.15, at which point they even out and remain constant thereafter. Scaling the model any more than 0.25 is waste of resources, as it does not provide any performance gains.

5.5 Final Model

We will now create an optimized model based on the results of the previous experiments. Firstly, our testing with data volume revealed that there is a consistent (although gradual) increase in performance when more data is provided; so, we will not exclude any training data for the final model. A dropout rate of 0.3—especially when combined with a smaller network size—is likely sufficient to prevent overfitting. To balance model performance with training time and memory usage, we will use a maximum of 5,000 features. Finally, we will scale down our original layer size by a factor of 4, for 128 and 64 neurons in the first and second layers respectively.

When initially evaluating this model, we find that its average performance metrics across all samples are 0.948 for precision, 0.718 for recall, and 0.807 for F_1. However, this varies widely between projects: *Biography* has 0.978/0.916/0.946, whereas *Geography*, which is lowest among all projects, has only 0.879/0.323/0.473. The difference between these two projects can be explained fairly easily: biography articles are typically in the format `[name] ([birth] - [death]) is/was a [nationality] [occupation]`. This makes them much easier for the model to identify. Geography articles tend to be much more diverse, and the *WikiProject Geography* tag tends to be applied to articles inconsistently.

A surprising result was that of *WikiProject United States*: its performance was only slightly better than that of Geography. To investigate this, a `wrong_where` method was added to the Model class. This function

returns articles from the dataset which were misclassified by the model. After examination, the cause of this appears to again be inconsistent tagging in the training dataset: there are often articles about an American topic, with "American" in the lead sentence, that are *not* tagged as part of the United States project. This results in the model often being unconfident and conservative for articles in this topic area.

We will make a modification to how labels are assigned. Up until now, the model generated probabilities for each project, which are then "predicted" if above a certain threshold (usually 0.5). Increasing this threshold generally increases precision and decreases recall, and vice versa. We implement a function `optimize_thresholds` which, for each class, determines a custom threshold to maximize its performance metrics.

The F_1 score, which considers both precision and recall equally, is a good metric for measuring a model's overall performance. However, for our application precision is more important: while not assigning WikiProjects to an article isn't a big deal (better something than nothing), assigning *incorrect* projects is disruptive (an editor will need to manually fix the mistake). Therefore we will instead use the F_β score, which allows us to set β depending on which metric we value more. We use $\beta = 0.5$, which strongly favours precision.

Overall, our model is extremely capable and accurate. It labels articles correctly in the majority of cases, and when it mislabels it is almost always due to the model's intentional conservative design, or flaws in the dataset itself, as opposed to being wrong. If expanded to label more than the 25 largest projects, this model could almost certainly be used in production on Wikipedia.

6 Conclusion

In this project, we successfully developed a deep feed-forward neural network to help volunteers address the significant backlog of unreviewed Wikipedia articles. By using TF-IDF vectorization on article leads, the model effectively automates the multi-label classification of new articles into relevant WikiProjects, working as a proof-of-concept to assist reviewers.

Through experimentation, we discovered several insights into model optimization. Firstly, we discovered that only a fraction of the available dataset is needed to make an accurate model; however, there was a consistent (albeit small) increase in performance with additional data. Our experiments with dropout and network size revealed that the initial network had a significant amount of excess capacity, allowing us to significantly reduce network layer sizes without impacting performance. Finally, we found that only around 5,000 features are needed to classify articles effectively. Together, these insights allowed us to decrease the network's trainable parameter count by a factor of 8 while keeping virtually all performance.

After creating our final model, we investigated project-specific misclassifications, such as the model's inaccuracy for the *Geography* project. This revealed that inconsistent human tagging in the training dataset was actively hindering the model. Because assigning incorrect projects is disruptive and creates manual work for editors, we implemented a custom metric to heavily prioritize precision over recall.

6.1 Next Steps

Moving forward, there a number of steps that could be taken to improve the model's performance, robustness, and scalability.

- While the simple technique of random undersampling was effective for the scope of our model, when scaling up the model to thousands of projects, many of which have only a few dozen articles, more advanced techniques may be needed. These could include partial random undersampling, which can use nearest neighbour heuristics to only remove irrelevant examples of majority classes, while keeping those closest to decision boundaries. This ensures that large class imbalances can be handled, while not removing valuable data.[4]

- To capture deeper semantic context, the relatively simple TF-IDF vectorizer could be replaced with pretrained word embeddings or transformer models, which would likely help with broad and inconsistently tagged topic areas like Geography.

[4] Nicolás García-Pedrajas, "Partial random under/oversampling for multilabel problems"

- In addition to WikiProjects, Wikipedia articles are classified into more specific categories (eg. *20th-century Canadian politicians*). This would require a much more complex model, and likely a different architecture entirely: while there are thousands of projects, there are millions of categories, with more created every day.

7 Code Organization

The code for this project is split between Python modules and Jupyter notebooks: testing, experiment, and data collection code is in notebooks, while the modularized `Model` class is in a Python file. The description for each file is as follows:

- `data_collection.ipynb` uses the steps described in Section 2 to collect data: it uses Wikipedia's API to collect articles in each project, find the lead text for each, find other WikiProjects for each article, and then saves the result to `all_data.csv`.
- `create_model.ipynb` walks through the steps for creating a model: using a binarizer to turn WikiProjects into vectors, splitting test and training data, vectorizing the text, and creating, training, and saving the model.
- `model.py` generalizes the code from this notebook to make a modular interface with a `Model` class. This allows us to pass in additional parameters to run experiments: dataset size, training duration, dropout, etc. It also provides `save`, `load`, and `get_report` functions, so we can easily tell how a model is performing, or load a previously trained model.
- `evaluation.ipynb` contains a function, `run_eval_thread`, which uses the Model class to create and train a model with the given parameters. It does this on a separate thread, to ensure memory is reclaimed once it is done. It then runs the model's `get_report` function, and returns statistics.
- `final_model.ipynb` contains our optimized model and code to evaluate it.

8 References

García-Pedrajas, N. (2024). Partial random under/oversampling for multilabel problems. Knowledge-Based Systems, 299, 112355. https://doi.org/10.1016/j.knosys.2024.112355

Olawale Awe, O. (2024). Computational strategies for handling imbalanced data in machine learning [Lecture notes/PowerPoint slides]. International Statistical Institute. https://isi-web.org/webinar/iasc-computational-strategies-tackling-imbalanced-data-machine-learning